

Optimal Instruction Block Structure for a Custom GPT

Custom GPT Instruction Blocks work best when they are well-structured, clearly segmented, and concise. The goal is to cover all necessary guidance (persona, goals, rules, style, etc.) in an organized way that the model can consistently follow. Below is an optimal structure with recommended section headers, typical content, and character budget allocation for each. We also discuss how upcoming features like `@mentions` and the use of **Referential Instruction Stub (RIS)** externalization can influence this structure.

Recommended Structure and Layout

A good Instruction Block is typically divided into **logical sections**, each with a specific purpose. For example, an effective prompt can be broken down into sections covering the AI's identity, its task or goal, interaction rules, style/tone, user guidance steps, and any final constraints or reminders ¹. Using Markdown headings (e.g. `#` or `##`) to label these sections greatly improves clarity – the model “recognizes these visual cues and adheres to your layout, enhancing readability and fidelity” ².

A commonly recommended structure (sometimes summarized by the acronym *INFUSE*) is as follows ¹:

- **Identity & Goal:** Define *who* the assistant is (persona/role) and *what* it is intended to achieve. This gives the model a clear persona and purpose up front ³. For example: “You are an expert financial advisor GPT whose goal is to help users plan budgets.”
- **Navigation/Interaction Rules:** Specify *how* the assistant should interact. This includes rules for using tools or knowledge sources, when to consult attached files, how to interpret commands, and any boundaries in engagement ⁴. For instance: when to use web browsing or a knowledge base, or “only answer questions within your domain.”
- **Flow & Personality (Tone/Style):** Set the desired communication style, tone, and any key personality traits ⁵. e.g. formal vs. friendly, use of technical language or simple explanations, emotive or neutral tone, etc. This keeps responses consistent in style.
- **User Guidance (Step-by-Step Instructions):** Provide a structured method for helping the user achieve their goal ⁶. This is often the largest section, outlining the approach the GPT should take. For example, if it's a troubleshooting assistant: step 1 – ask clarifying questions; step 2 – suggest solutions; step 3 – verify resolution, etc. Breaking down tasks into triggers and instructions (with delimiters) can improve reliability ⁷ ⁸.
- **Signals & Adaptation (Optional Advanced):** If needed, instruct how to adjust responses based on user cues or emotional signals ⁹. For instance, “If the user seems confused, simplify the explanation.” (This can also be handled via examples or separate files, as noted below.)
- **End Instructions (Final Reminders/Constraints):** Reinforce any critical rules the model *must not forget* ¹⁰. This might include don't-do or always-do rules (e.g. “Never provide medical advice” or “Always end with a summary”) and any final checks (like reminding the model to double-check output for completeness). Placing these as a concluding section helps keep the model aligned with its purpose until the end ¹⁰.

Example: A simple structured instruction might look like:

```
# Role and Goal
You are **ChefBot**, a culinary assistant GPT. Your goal is to help users create meals from available ingredients.

# Interaction Rules
- If the user provides a list of ingredients, suggest a suitable recipe.
- If asked for cooking techniques, refer to *CookingTips.txt* (attached knowledge file).
- Do not provide medical or allergen advice (not in your scope).

# Tone and Style
Use a friendly, encouraging tone. Explain steps clearly in simple terms. Avoid technical jargon.

# Step-by-Step Guidance
1. Greet the user and confirm their objective or available ingredients.
2. Propose a recipe or meal plan that fits their input.
3. Offer step-by-step cooking instructions.
4. Ask if they need further help or have dietary restrictions.

# Final Checks
Before answering, **review your suggestion** for completeness and accuracy.
Ensure the recipe uses the ingredients given and the instructions are in order.
```

This structured layout makes it easy for the model to follow along. OpenAI's documentation also shows using headings like “# Context”, “# Instructions”, “# Additional Information” to segment different parts of the prompt ¹¹ ¹². In practice, you can customize section titles to fit your GPT's needs, but the key is to cover the **who**, **what**, **how**, **style**, and **must-nots** in a clear order.

Effective Section Headers

Using descriptive **section headers** in your instruction block improves the clarity of the prompt and thus the quality of responses. Good headers act as guideposts for the model. Based on best practices, the following headers (or similar wording) tend to yield strong results:

- **“Role” or “Identity”** – to establish the AI's persona and role.
- **“Goal” or “Objective”** – to state the primary mission or problem to solve (often combined with Role as a single section, e.g. “Role & Goal”).
- **“Instructions” or “Tasks”** – to enumerate what the assistant should do, often as steps or rules. You might break this further into sub-sections like “Core Tasks” or “Procedure”.
- **“Knowledge & Tools”** – if the GPT has access to files or web, a section telling it when/how to use those resources (this could also be part of Navigation Rules). For example, “Use **ProjectSpecs.pdf** for technical details when relevant.”

- **“Tone/Style Guidelines”** – to lock in the desired writing style or tone (could be combined under a broader **“Persona & Style”** header).
- **“Capabilities and Limitations”** – an explicit list of what the GPT *can do* versus *must not do* ¹³ ¹⁴ . This helps keep the model’s output on-brand and safe.
- **“Examples” or “Formatting” (if needed)** – providing a few-shot example or specifying output format. For instance, a **“Output Format”** section can show the model how to structure its answers (bullet points, JSON, etc.), which it will mimic.
- **“Final Check” or “End Instructions”** – a reminder for final behavior checks (e.g. “Review your answer for errors before responding” ¹⁵ ¹⁶ , or reiterating a do-not-violate rule).

These headers are phrased in a way that is intuitive. They align with guidance to **use Markdown headings to organize content**, which improves the model’s adherence to the instructions ² . For example, starting with a persona priming line under a **Role** header, or listing rules under an **Instructions** header, has been shown to keep the model on script by clearly delineating each aspect of its programming ¹¹ . You should choose header names that make sense for your use case, but the ones above are proven to work well by clearly signaling the content of each section.

Allocation of Character Budget per Section

The Instruction Block has a limited total length (OpenAI notes a request prompt must stay under ~100,000 characters, with practical shorter limits for the system message) ¹⁷ . It’s important to allocate this “character budget” wisely across the sections. While there is no rigid rule, a **rough percentage breakdown** of the instruction content for each section is as follows:

- **Identity/Role & Goal:** *Approximately 10%* of the characters. This section should be succinct – usually a few lines to establish context. It’s crucial but doesn’t need to be long; a concise description of the persona and mission is enough.
- **Interaction Rules (including Tools/Knowledge usage):** *Around 15–20%*. These are the guidelines for how to act and use resources. They might be a list of rules or triggers, which can take a moderate portion of the prompt. If your GPT has many tools or complex rules, this might grow, but generally keep each rule concise.
- **Tone & Style Guidelines:** *About 10%*. Defining tone or style typically takes only a few lines or bullet points (e.g. “Tone: polite and technical; avoid jokes”). This tends to be a smaller section, but it’s important for consistency.
- **User Guidance Steps/Tasks:** *The largest chunk, roughly 30–40%*. This is the step-by-step process or the heart of the instructions for fulfilling user requests. Since it may contain multiple steps or detailed approaches, it naturally occupies a big portion. For complex GPT behaviors, this could even be up to ~40% of the content. It’s where you ensure the GPT knows *how* to help the user in detail.
- **Examples/Formatting (if included):** *Varies, perhaps ~10%*. If you include a few-shot example or explicit output format instructions, allocate a modest portion. Often one good example or a small table of “Bad vs Good response” suffices ¹⁸ ¹⁹ . Keep examples brief since they count toward the token budget.
- **Constraints & Final Reminders:** *~10%*. This includes the “End Instructions” or must-not break rules at the end. Bullet-point any critical don’ts (or always-do’s) here. It shouldn’t be too lengthy – just the key points you absolutely need to reinforce (safety instructions, never revealing certain info, etc.). Even a short final note has impact, as it is the last thing the model “hears” in the system prompt.

Keep in mind these percentages are approximate. For instance, a simpler GPT might not need a separate examples section at all (freeing that budget for more user-guidance content), or a particularly complex tool usage might make *Interaction Rules* section a bit larger. The guiding principle is to **prioritize the sections that directly help the model perform its task**. In many cases, the step-by-step guidance and core instructions deserve the most space, whereas persona, tone, and final reminders can be conveyed in a few well-chosen sentences. Always aim for clarity and brevity – every extra word is part of the model's context load. OpenAI's guidance echoes this: break complex instructions into clear, granular steps and don't let less important details crowd out the critical instructions ²⁰ ²¹ .

Impact of @mention (Multi-GPT) on Structure

The @mentions feature (allowing multiple custom GPTs in one conversation) introduces multi-agent interactions. **Planning for @mention does not drastically change the overall instruction block structure**, but it does require a few adjustments in emphasis:

- **Responding Only When Addressed:** In a multi-GPT scenario, each GPT should ideally respond only to prompts directed at it. You may want to include a note in the Interaction Rules like, *"If the conversation is directed to another assistant, remain silent unless called upon."* This ensures the GPT doesn't intrude when it's not tagged. (The platform's mention system largely handles this routing automatically, but an explicit rule can provide a safety net.)
- **Consistent Shared Context:** If you use multiple GPTs that hand off tasks via @mention, they might need shared understanding of certain context or style. **Ensure any common protocols or persona traits are defined in each GPT's instructions** (or leveraged via shared external files, as discussed below). OpenAI's best practice is that if you plan to have GPT-A call GPT-B, the relevant context should persist – the conversation history will carry over, but each GPT will still use its own instruction set. If you want them to maintain a unified style or knowledge, **make that part of each GPT's config**. For example, both could include a similar "Tone & Formatting" section so they don't conflict in voice.
- **Using RIS for Shared Logic:** Notably, the RIS system is *complementary* to @mention handoffs: *"When GPT-A calls GPT-B using @mention, the thread carries context – but files remain those of GPT-B. If each GPT has identical RIS files, shared logic (tone, rules) continues seamlessly across agents."* ²² In practice, this means if you know GPTs will interoperate, you can externalize their common instructions to the same attached files (via RIS) so that even when switching bots, the underlying persona or rules stay in sync. **Consistency is key** in multi-GPT workflows. As the RIS guidelines note, you should stub out *shared* behaviors into files, but keep any *unique* behavior inline for each GPT ²³ . This way, anything meant to be universal (like a corporate style guide, or a specific role trait) persists across mentions, while things that differentiate one GPT from another remain in that GPT's own prompt.

In summary, enabling @mentions doesn't require a new section per se, but you should review your **Navigation/Interaction Rules** section to account for multi-agent context. Add a brief rule about only responding when addressed, and use consistent instruction design (or shared RIS files) across your GPTs if you expect them to collaborate. These tweaks ensure that using @mention won't break the flow – each GPT will play its part without confusion.

Impact of RIS (Referential Instruction Stub) on Structure

Incorporating **RIS concepts** (Referential Instruction Stubs) can significantly *modify how you structure the instructions*, mainly by offloading some sections out of the main prompt. RIS is essentially a way to move portions of your instruction block (especially long, persistent or repetitive parts) into external attached files and reference them with a short pointer. This frees up space in the main Instruction Block and ensures consistency across GPTs ²⁴ ²⁵ .

Using RIS will change the composition of your Instruction Block in these ways:

- **Offloaded Sections:** Typically, you would offload any “*persistent, non-changing logic*” – for example, extensive tone descriptions, persona details that are constant, or standard formatting rules. Instead of writing these out in full, you include a stub reference in the prompt that tells the GPT to load that info from an attached file ²⁴ ²⁶ . For instance, rather than a 300-character paragraph defining the persona's entire background and tone, you might have a one-liner like `referential_stubs: - persona: persona_profile_v1.md::FriendlyAdvisor` in the instructions.
- **Shorter Main Prompt Sections:** The sections that have been externalized via RIS will shrink dramatically in the main prompt. A chunk that might have been, say, 500 characters of inline text can be replaced by a ~30 character stub reference ²⁷ . That's a huge reduction – on the order of a **90–95% character savings** for that piece of content. For example, inline tone rules that cost 250–800 characters can be substituted with a stub costing only ~30 characters ²⁸ . So your **“Role/Persona” section and perhaps part of “Tone & Style” might become just a couple of stub lines** instead of long descriptions.
- **Structure Remains, content shifts:** You will still want to maintain logical sections in your instruction layout, but some sections will mostly contain pointers to files rather than full text. It's good practice to keep a clear header and a note that the logic is referenced externally (for transparency and easy editing). For example, you might still have a **Role** heading, but under it: “(Using external persona profile via RIS)” followed by the stub reference. This lets you preserve the overall outline (so the GPT knows the sections), while delegating details to files. Sections like **User Guidance/Task steps** that are dynamic and task-specific would likely remain inline (those you typically wouldn't offload, since they are core to this GPT's unique function). Sections that *can* be common templates – like generic tone enforcement or common safety rules – are prime candidates for RIS externalization.
- **Cross-GPT Alignment:** As mentioned, if you have multiple GPTs sharing the same RIS files (for a common persona or style), each GPT's instruction block can be lighter. The **RIS architecture is designed for “single-source-of-truth”** for shared logic ²⁵ , so you write it once in a file instead of duplicating across prompts. This means the structure of any one GPT's prompt might omit certain details (since they're in the file), but as a whole system your GPTs are aligned. Just be mindful to **not externalize truly unique instructions** for a GPT into a shared file – keep unique behaviors inline (as the RIS guide warns, avoid stubbing out unique behavior per GPT ²⁹).

Extent of Modification: In practical terms, using RIS can shift the “percentage of content” each section occupies in the main prompt. For instance, before RIS you might allocate ~10% of characters to identity/tone. After implementing RIS, that section in the prompt might drop to only ~2–3% (just a stub entry),

because the detailed content lives in a file. The freed 7–8% can then be reallocated to other sections like more step-by-step guidance or additional examples. Similarly, a set of lengthy formatting rules could be moved out, shrinking what was say 15% of the prompt to under 5%. Overall, your **main instruction becomes more lightweight** and focused on task-specific guidance, while the static background info is pulled in on demand.

The net benefit is a cleaner prompt and more room for customization. However, note that using RIS assumes the platform supports those stub references (it's a feature of custom GPT projects and file attachments). When implemented properly, **the model will fetch the external instructions “without occupying instruction block real estate”** ³⁰ ³¹. This can greatly improve maintainability and ensure you aren't hitting length limits by repeating the same boilerplate in every GPT.

Additional Instruction Capacity via Externalization

By employing externalization techniques like RIS (or generally attaching large knowledge files), you can effectively **increase the total instructions/knowledge the model can utilize** without exceeding the core prompt character limit. In other words, you gain additional instruction capacity beyond the normal inline character budget.

The exact *percentage of additional characters* you can leverage depends on how much content you move out-of-band, but it can be quite significant. Here are some considerations:

- **Character Savings:** As noted, each stub can save hundreds of characters. For example, replacing a 500-character inline chunk with a 30-character reference frees ~470 characters in the prompt (~94% of that chunk) ²⁷. If your instruction block initially was near the limit (say 8,000 characters for GPT-4 context), offloading a few such chunks could free on the order of **15–25% of that space**. You can then use that freed space to add more instructions or examples that wouldn't have fit before. In percentage terms, you might transform an instruction block that was maxed out into one that is only ~75–85% of the limit, leaving an extra 15+% capacity for new content.
- **Leveraging Large Files:** OpenAI's custom GPTs allow attaching up to 20 files of 512MB each ³². While the model obviously can't ingest gigabytes of text at once, these files can be queried as needed. Effectively, **you can offload a huge amount of reference material or rarely-used instructions** into these files. The main prompt just needs to include cues for *when to use which file*. This means the model's “knowledge/instruction pool” is much larger than the prompt alone. For example, you might only dedicate 5% of your prompt to instruct the GPT *how* to use external knowledge (which file to consult for what), yet you might have thousands of additional characters of actual content in those files at its disposal. This isn't a straight addition to prompt length, but functionally it **extends what the GPT can reference** far beyond the normal character limit – potentially by **50% or more** in terms of useful instruction content. (Many users find they can pack in far more detail via attached files than they ever could in the prompt.)
- **Safe Estimates:** If we speak strictly about *instruction text*, a conservative estimate is that externalization lets you apply at least an **additional 20–30% of instruction detail** beyond what you could if everything had to be inline. In some cases, the figure is higher. For instance, if your original prompt was 6,000 characters of instructions and you managed to move 1,500 of those into RIS files (25% of it), you've freed up that 1,500 characters to use elsewhere – effectively allowing 1.25× the

original detail. If you have even more boilerplate (persona descriptions, extensive style rules) externalized, you might push that to 1.5× (50% more) without hitting limits.

In summary, externalizing via RIS or knowledge files **increases the effective instruction payload** significantly. It preserves the core 100k character limit for requests ¹⁷ by keeping your prompt lean, while still bringing in rich instructions from outside. Just be sure to use this extra capacity wisely – more characters should go toward **enhancing clarity or providing useful examples**, not just adding fluff. The combination of a well-structured core prompt and expansive external references is powerful: you get comprehensiveness without sacrificing coherence. As the RIS system highlights, it “reduces 500+ characters of logic down to ~30” per reference ²⁷, allowing you to pack in **considerably more instructional content** overall than a monolithic prompt would allow. This means you can confidently allocate that saved space (often on the order of tens of percent of your prompt budget) to make your Custom GPT even more capable and specific.

Sources:

- OpenAI, *Key Guidelines for Writing Instructions for Custom GPTs* ¹¹ ¹² (example of structured prompt with sections)
- Chen Fang, *Towards AI – Custom GPT context stability* (INFUSE prompt sections) ¹ ³
- *BecomeAI Marketer – 10 Hacks for Killer GPTs* (markdown formatting and persona tips) ² ¹³
- OpenAI Community, *Referential Instruction Stub (RIS) Explainer* ²² ²⁷ (on using RIS with mentions and character savings)
- Max (PromptGenius), *How to Build Custom GPTs (2025)* ⁴ ¹⁰ (on interaction rules, end instructions)

¹ ³ ⁴ ⁵ ⁶ ⁹ ¹⁰ How To Build CustomGPTs -- 2025 Guide : r/ChatGPTPromptGenius
https://www.reddit.com/r/ChatGPTPromptGenius/comments/1j2v124/how_to_build_customgpts_2025_guide/

² ⁷ ⁸ ¹³ ¹⁴ ¹⁵ ¹⁶ ¹⁸ ¹⁹ The Secret Sauce for Cooking Up Killer GPTs
<https://www.becomeanaimarketer.com/p/the-secret-sauce-for-cooking-up-killer-gpts>

¹¹ ¹² ²⁰ ²¹ Key Guidelines for Writing Instructions for Custom GPTs | OpenAI Help Center
<https://help.openai.com/en/articles/9358033-key-guidelines-for-writing-instructions-for-custom-gpts>

¹⁷ ³² Build Custom GPTs with Oxylabs' Web Scraper API
<https://oxylabs.io/resources/integrations/custom-chatgpt>

²² ²³ ²⁴ ²⁵ ²⁶ ²⁷ ²⁸ ²⁹ ³⁰ ³¹ RIS_ReferentialInstructionStub_Explainer_v1.0.md
<file:///file-3mH1H2KVQ8j5nZjiUt9wGC>